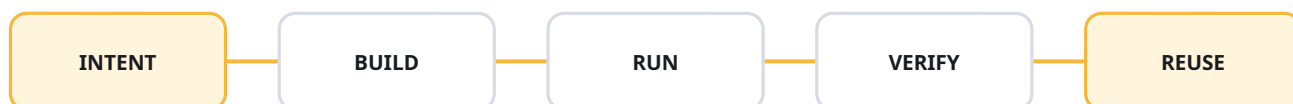


Technical Vision & Architecture

From probabilistic reasoning to reliable, reusable automation.

Use intelligence to understand, build, diagnose, verify and repair. Then preserve what works as reusable deterministic behavior.



Reason when needed. Preserve what works.

DRAFT V0.9

October 2026

Grounded in the supplied FluxIQ dev and FluxIQWebExtension dev snapshots.

The core thesis

FluxIQ is designed to move work from probabilistic reasoning into reusable deterministic software.

The problem is not that agents cannot act.

The problem is that many agentic systems keep paying for reasoning on every run, even after they have already learned what successful behavior looks like.



FluxIQ treats intelligence as a builder, diagnostician and judge.

The Flow is the durable artifact. Model reasoning is concentrated where ambiguity exists, then stable behavior is executed through explicit runtime primitives.

DESIGN PRINCIPLE

Stable behavior should migrate toward deterministic execution.

What exists today

The current codebase already contains the core pieces of the build, run, verify and repair loop.

STATUS

IMPLEMENTED

NEAR COMPLETE

IN PROGRESS

VISION

● Instruction-first authoring	A flow bootstrap can begin from an active instruction and a capability-filtered node catalog.	IMPLEMENTED
● Typed automation runtime	Flow contracts, DSL/compiler, native nodes, control flow and runtime sessions are present.	IMPLEMENTED
● Browser execution layer	Deterministic actions, recording, extraction, page evidence and recovery signals.	NEAR COMPLETE
● Result verification	Selected outcomes can be judged against the requested result after execution.	IMPLEMENTED
● Repair and re-authoring	Refuted outcomes can route into repair logic, rerun and be judged again.	IN PROGRESS
● Persistence and governance	Project-scoped persistence, permissions, versions, conversations and adaptation history.	IN PROGRESS

These labels are intentionally conservative. Code presence does not mean every path is production hardened.

Current architecture

A domain-neutral control plane with pluggable execution environments.



CORE

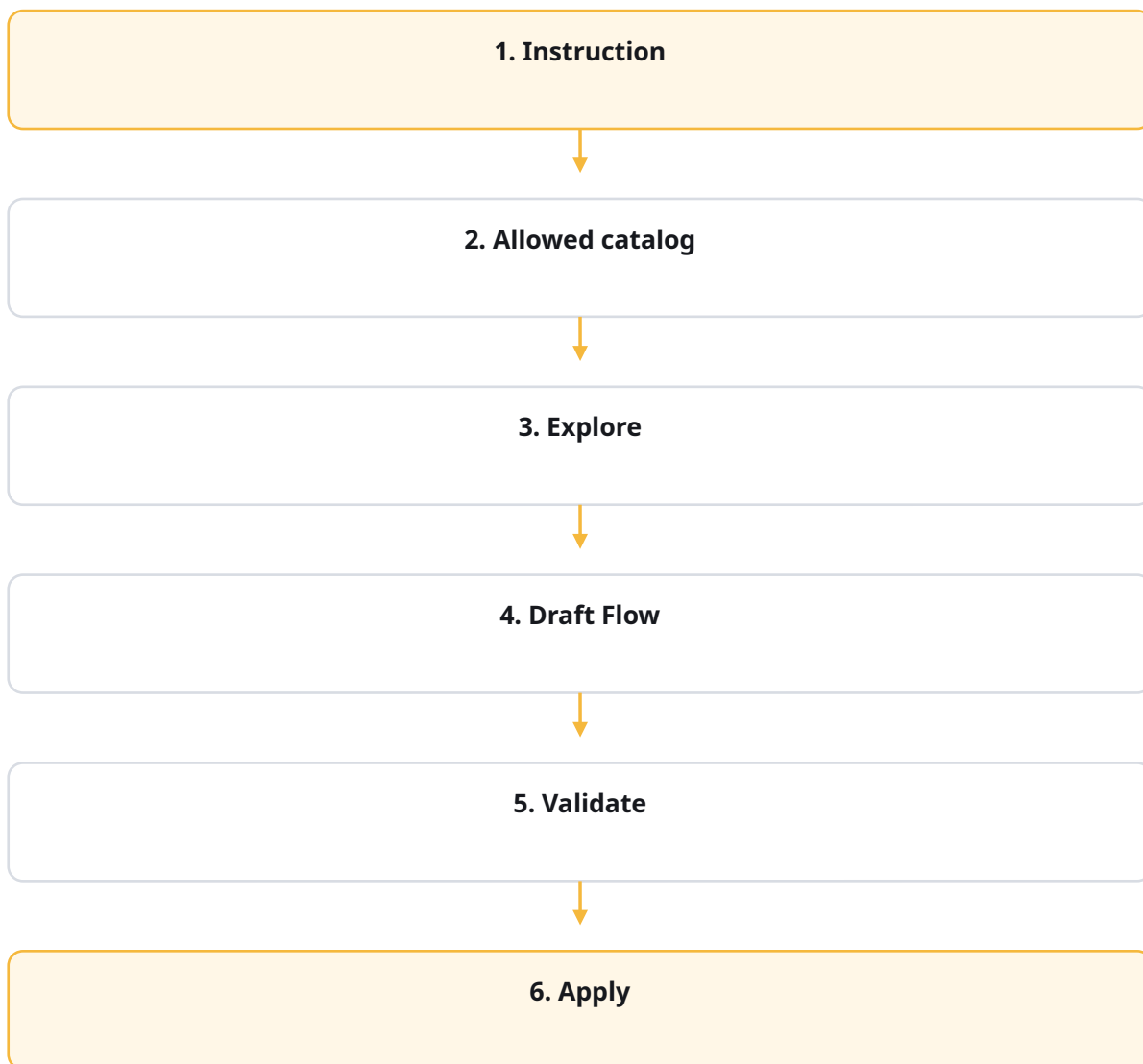
Automation Studio and control plane for authoring, lifecycle, persistence, verification, adaptation and orchestration.

DOMAINS

Browser today, other environments later. Host projects register domain state, actions, inputs, outputs and runtime adapters.

From instruction to executable Flow

The model composes known capabilities instead of inventing arbitrary actions.



Capability scope

Only registered, available primitives enter the authoring context.

Permission filtering

Consequential actions can be withheld independently of the user instruction.

Explicit refusal

Authoring can refuse when context or capabilities are insufficient.

OBSERVED

A prior end-to-end development run generated a deterministic Flow for about \$0.016 in model cost. One observed run, not a universal price claim.

Deterministic first, intelligence when needed

Browser target resolution is layered to absorb superficial drift before escalating.



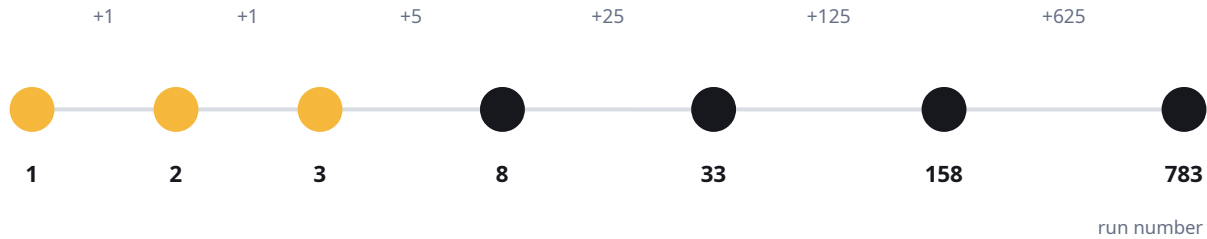
REFUSAL IS A FEATURE

Exact matches can be rejected when identity evidence disagrees. Low-confidence or ambiguous candidates are refused instead of blindly selecting the nearest element.

Confidence should change cost

Verify heavily when behavior is new or changed, then check less often as confidence accumulates.

Verification schedule



Graph changes reset confidence

A changed Flow returns to the early verification window.

Refutations force attention

The next run is checked.
Repaired reruns are checked too.

Stable behavior gets cheaper

The verification interval grows as confidence accumulates.

CORE IDEA

Do not keep paying the same intelligence cost after a Flow has repeatedly proved itself.

Verification feeds repair

A result judgement is operational. It can change what happens next.



ADAPTIVE LOOP

The endpoint is not an agent that improvises forever. It is automation that brings intelligence back in when reality invalidates what it learned.

Economics of intelligence

The relevant metric is amortized intelligence cost per successful automated outcome.

AUTHOR**Spend reasoning to create a reusable artifact**

high value

EXECUTE**Prefer deterministic actions and evidence-based resolution**

low recurring cost

VERIFY**Judge frequently at first, then decay the checks**

declining cost

REPAIR**Spend again only when the artifact is invalidated**

event-driven cost

NORTH-STAR METRIC**Amortized intelligence cost per successful outcome across the lifetime of a Flow.**

From flows to generated applications

The long-term direction is to generate the software around an outcome, not only a single workflow.

VISION

Long-term platform direction

USER INTENT

“Build me a lead-generation application for commercial roofing companies in Western Canada.”



Interface

operator and customer interfaces

Data

schemas, records and provenance

Flows

discovery, enrichment, qualification and refresh

Integrations

browser, APIs, messaging and databases

Governance

permissions, verification, approvals and rollback

Generated applications should stay inspectable. Flows, data contracts, permissions and adaptations remain explicit artifacts.

Application generation is the long-term direction, not a claim about the current product.

Roadmap and evaluation

The architecture should earn its claims through measured runtime behavior.



NOW

Harden the browser loop

Flow creation, evidence, verification, repair, UI and end-to-end testing.



NEXT

Cloud deployment and safe adaptation

Let users deploy adapting flows to the cloud, keep them running continuously and persist validated repairs safely.



THEN

Integrations and composition

Cross-flow composition, cloud integrations, versioning, debugging and reusable components.



LATER

Generate applications

Coordinated flows, data, interfaces, permissions, integrations and maintenance behavior.

What to measure

Real task completion

Recovery rate

Deterministic reuse

False-success rate

Latency

Model spend

Human intervention

EVALUATION TEST

Does added intelligence create a more reliable reusable artifact, or only more reasoning that must be paid for forever?

Implementation evidence reviewed

Representative architecture and source areas reviewed in the supplied dev snapshots.

CORE

README • current-system.md • automation-studio.md • runtime-kernel.md • flow bootstrap + LLM handlers

RUNTIME

flow DSL/compiler • native/control-flow/policy nodes • result-check scheduling • result verification • graph judgement persistence

BROWSER

web-capabilities.md • element-identity.md • build-loop.md • runtime actions • automated test facility

ADAPTATION

refuted-result recovery • adaptation persistence • runtime adaptation tests • working adaptive-flow roadmap

Approximate source inventory

Core package about 2,059 TypeScript source files. Core/web about 963 test/spec files. Web Extension about 2,597 TypeScript files and about 735 test/spec files.

Counts are descriptive only, not quality metrics. This is a technical vision document, not an independent security audit or production-readiness certification.